МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №4

Введение в нейронные сети:
построение многослойного перцептрона

Специальность системы управления информацией

Выполнил
М. А. Хомич,
студент группы АС-68/24
Проверил
А. А. Крощенко,
доцент Кафедры ИИТ,
«__» _____2026 г.

Брест 2026

Цель: построить, обучить и оценить многослойный перцептрон (MLP) для решения задачи классификации, а также сравнить качество двух архитектур нейронной сети.

Вариант 10. Adult Census Income.

Задача: предсказать, превышает ли годовой доход человека 50 тыс. долларов США (бинарная классификация).

Краткое описание хода работы

1. Загружен датасет Adult Census Income (32 561 запись, 15 столбцов). Пропуски, обозначенные символом «?», заменены на NaN. Числовые признаки заполнены медианой и стандартизированы (StandardScaler), категориальные — наиболее частым значением и закодированы методом One-Hot Encoding. Данные разделены на обучающую (75 %) и тестовую (25 %) выборки со стратификацией по целевому классу. После кодирования получено 104 входных признака.

2. Реализован класс MLP на основе torch.nn.Module. Базовая архитектура содержит один скрытый слой на 32 нейрона с активацией ReLU и выходной слой с одним нейроном. Для бинарной классификации используется функция потерь BCEWithLogitsLoss, объединяющая Sigmoid и бинарную кросс-энтропию; оптимизатор — Adam с learning rate 0,001.

3. Обучение выполнялось 30 эпох с размером батча 256. После каждой эпохи рассчитывались потери на обучающей и тестовой выборках. В рамках эксперимента построена более глубокая сеть с двумя скрытыми слоями по 16 нейронов и проведено сравнение метрик Accuracy, Precision, Recall и F1-score для класса «доход > 50K».

Код программы:

```
"""
Лабораторная работа №4.
Введение в нейронные сети: построение многослойного перцептрона.


Вариант 10 — Adult Census Income.
Задача: предсказать, превышает ли доход человека $50 тыс. в год
(бинарная классификация).
```

```
Базовая архитектура:  один скрытый слой на 32 нейрона (ReLU),
                       выходной слой с 1 нейроном.
Эксперимент:          более глубокая сеть — два скрытых слоя по 16 нейронов.
"""


import os
import random
import time


import matplotlib
matplotlib.use("Agg")  # сохранение графиков без открытия окна
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.utils.data import DataLoader, TensorDataset


from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.metrics import (
    ConfusionMatrixDisplay,
    accuracy_score,
    classification_report,
    confusion_matrix,
    f1_score,
    precision_score,
    recall_score,
)
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler


# ---------------------------------------------------------------------------
# Пути к файлам (работают при запуске из любой папки)
# ---------------------------------------------------------------------------
SRC_DIR = os.path.dirname(os.path.abspath(__file__))
REPORT_DIR = os.path.abspath(os.path.join(SRC_DIR, "..", "rep"))
DATA_PATH = os.path.join(SRC_DIR, "adult.csv")
os.makedirs(REPORT_DIR, exist_ok=True)


# ---------------------------------------------------------------------------
# Воспроизводимость результатов
# ---------------------------------------------------------------------------
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)


DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Лабораторная работа №4")
print("Вариант 10 — Adult Census Income")
print("Устройство для вычислений:", DEVICE)
print("Версия PyTorch:", torch.__version__)



# ---------------------------------------------------------------------------
# 1. Импорт данных и предобработка
# ---------------------------------------------------------------------------
def load_and_prepare_data():
    """Загружает датасет, кодирует признаки и возвращает тензоры."""
    df = pd.read_csv(DATA_PATH)
```

```
    # Пропуски в исходном датасете обозначены знаком '?'
    df = df.replace("?", np.nan)


    print("\nРазмер набора данных:", df.shape)
    print("Количество пропусков:", int(df.isna().sum().sum()))
    print("Распределение целевой переменной:")
    print(df["income"].value_counts().to_string())


    X = df.drop(columns=["income"])
    # 1 — доход > 50K, 0 — доход <= 50K
    y = (df["income"].astype(str).str.strip() == ">50K").astype(np.float32)


    numeric_cols = X.select_dtypes(include=np.number).columns.tolist()
    categorical_cols = X.select_dtypes(exclude=np.number).columns.tolist()


    numeric_pipeline = Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
    ])
    categorical_pipeline = Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(handle_unknown="ignore", sparse_output=False)),
    ])
    preprocessor = ColumnTransformer([
        ("num", numeric_pipeline, numeric_cols),
        ("cat", categorical_pipeline, categorical_cols),
    ])


    # Разделение: 75 % — обучение, 25 % — тестирование
    X_train_raw, X_test_raw, y_train_raw, y_test_raw = train_test_split(
        X, y, test_size=0.25, random_state=SEED, stratify=y
    )


    # Обучаем преобразования только на train, чтобы избежать утечки данных
    X_train = preprocessor.fit_transform(X_train_raw).astype(np.float32)
    X_test = preprocessor.transform(X_test_raw).astype(np.float32)


    y_train = y_train_raw.to_numpy(dtype=np.float32).reshape(-1, 1)
    y_test = y_test_raw.to_numpy(dtype=np.float32).reshape(-1, 1)


    X_train_tensor = torch.tensor(X_train, dtype=torch.float32)
    X_test_tensor = torch.tensor(X_test, dtype=torch.float32)
    y_train_tensor = torch.tensor(y_train, dtype=torch.float32)
    y_test_tensor = torch.tensor(y_test, dtype=torch.float32)


    print("\nОбучающая выборка:", tuple(X_train_tensor.shape))
    print("Тестовая выборка:", tuple(X_test_tensor.shape))
    print("Число входных признаков после кодирования:", X_train.shape[1])
    print("Доля доходов >50K в обучении:", round(float(y_train.mean()), 4))


    return (
        X_train_tensor,
        X_test_tensor,
        y_train_tensor,
        y_test_tensor,
    )



# ----------------------------------------------------------------------------
# 2. Архитектура многослойного перцептрона
```

```
# ---------------------------------------------------------------------------
class MLP(nn.Module):
    """Полносвязная нейронная сеть с настраиваемой архитектурой."""


    def __init__(self, input_size, hidden_sizes, activation=nn.ReLU):
        super().__init__()
        layers = []
        previous_size = input_size
        for neurons in hidden_sizes:
            layers.append(nn.Linear(previous_size, neurons))
            layers.append(activation())
            previous_size = neurons
        # Выходной слой: 1 нейрон (логит бинарной классификации)
        layers.append(nn.Linear(previous_size, 1))
        self.network = nn.Sequential(*layers)


    def forward(self, x):
        return self.network(x)



# ---------------------------------------------------------------------------
# 3. Обучение модели
# ---------------------------------------------------------------------------
def train_model(model, X_train, y_train, X_test, y_test,
                epochs=30, batch_size=256, learning_rate=0.001):
    """Цикл обучения с оценкой потерь на train и test после каждой эпохи."""
    loader = DataLoader(
        TensorDataset(X_train, y_train),
        batch_size=batch_size,
        shuffle=True,
        generator=torch.Generator().manual_seed(SEED),
    )
    # BCEWithLogitsLoss = Sigmoid + Binary Cross-Entropy (численно устойчивее)
    criterion = nn.BCEWithLogitsLoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)


    history = {"train_loss": [], "test_loss": []}
    start_time = time.perf_counter()


    for epoch in range(epochs):
        model.train()
        total_loss = 0.0
        for xb, yb in loader:
            xb, yb = xb.to(DEVICE), yb.to(DEVICE)


            y_pred = model(xb)              # forward pass
            loss = criterion(y_pred, yb)    # функция потерь


            optimizer.zero_grad()           # обнуление градиентов
            loss.backward()                 # обратное распространение
            optimizer.step()                # шаг оптимизации


            total_loss += loss.item() * xb.size(0)


        history["train_loss"].append(total_loss / len(loader.dataset))


        model.eval()
        with torch.no_grad():
            test_logits = model(X_test.to(DEVICE))
            test_loss = criterion(test_logits, y_test.to(DEVICE)).item()
        history["test_loss"].append(test_loss)
```

```
        if (epoch + 1) % 5 == 0 or epoch == 0:
            print(
                f"Эпоха {epoch + 1:2}/{epochs}: "
                f"ошибка обучения = {history['train_loss'][-1]:.4f}, "
                f"ошибка теста = {test_loss:.4f}"
            )

    elapsed = time.perf_counter() - start_time
    return history, elapsed



# ---------------------------------------------------------------------------
# 4. Оценка качества
# ---------------------------------------------------------------------------
def evaluate_model(model, X_test, y_test):
    """Считает вероятности, классы и метрики на тестовой выборке."""
    model.eval()
    with torch.no_grad():
        logits = model(X_test.to(DEVICE))
        probabilities = torch.sigmoid(logits).cpu().numpy().ravel()

    predictions = (probabilities >= 0.5).astype(int)
    real = y_test.cpu().numpy().ravel().astype(int)

    metrics = {
        "Accuracy": accuracy_score(real, predictions),
        "Precision": precision_score(real, predictions, zero_division=0),
        "Recall": recall_score(real, predictions, zero_division=0),
        "F1-score": f1_score(real, predictions, zero_division=0),
    }
    return metrics, predictions, real



# ---------------------------------------------------------------------------
# 5. Графики
# ---------------------------------------------------------------------------
def plot_loss_curves(history_basic, history_deep, epochs):
    fig, axes = plt.subplots(1, 2, figsize=(13, 4))
    for ax, history, title in zip(
        axes,
        [history_basic, history_deep],
        ["Базовая MLP: 32 нейрона", "Углублённая MLP: 16 + 16 нейронов"],
    ):
        ax.plot(range(1, epochs + 1), history["train_loss"], label="Обучение")
        ax.plot(range(1, epochs + 1), history["test_loss"], label="Тест")
        ax.set(title=title, xlabel="Эпоха", ylabel="BCE-потери")
        ax.grid(alpha=0.3)
        ax.legend()

    plt.tight_layout()
    path = os.path.join(REPORT_DIR, "lab4_loss_curves.png")
    plt.savefig(path, dpi=300)
    plt.close(fig)
    print("Сохранён график:", path)



def plot_confusion_matrices(pred_basic, pred_deep, real):
    fig, axes = plt.subplots(1, 2, figsize=(12, 5))
    for ax, prediction, title in zip(
        axes,
        [pred_basic, pred_deep],
        ["Базовая (32)", "Углублённая (16 + 16)"],
    ):
```

```
        cm = confusion_matrix(real, prediction, labels=[0, 1])
        ConfusionMatrixDisplay(
            confusion_matrix=cm,
            display_labels=["≤50K", ">50K"],
        ).plot(cmap="Blues", colorbar=False, ax=ax, values_format="d")
        ax.set_title(title)

    plt.tight_layout()
    path = os.path.join(REPORT_DIR, "lab4_confusion_matrices.png")
    plt.savefig(path, dpi=300)
    plt.close(fig)
    print("Сохранён график:", path)



def plot_metrics_comparison(comparison):
    metrics = ["Accuracy", "Precision", "Recall", "F1-score"]
    x = np.arange(len(metrics))
    width = 0.35


    fig, ax = plt.subplots(figsize=(9, 5))
    ax.bar(x - width / 2, comparison.loc[metrics, "Базовая (32)"],
           width, label="Базовая (32)")
    ax.bar(x + width / 2, comparison.loc[metrics, "Углублённая (16 + 16)"],
           width, label="Углублённая (16 + 16)")


    ax.set_xticks(x)
    ax.set_xticklabels(metrics)
    ax.set_ylim(0, 1)
    ax.set_ylabel("Значение метрики")
    ax.set_title("Сравнение качества моделей (класс >50K)")
    ax.grid(axis="y", alpha=0.3)
    ax.legend()


    plt.tight_layout()
    path = os.path.join(REPORT_DIR, "lab4_metrics_comparison.png")
    plt.savefig(path, dpi=300)
    plt.close(fig)
    print("Сохранён график:", path)



# ---------------------------------------------------------------------------
# 6. Основной сценарий
# ---------------------------------------------------------------------------
def main():
    (
        X_train_tensor,
        X_test_tensor,
        y_train_tensor,
        y_test_tensor,
    ) = load_and_prepare_data()


    input_size = X_train_tensor.shape[1]


    # Базовая архитектура по заданию варианта
    model_basic = MLP(input_size, hidden_sizes=[32]).to(DEVICE)
    # Эксперимент: более глубокая сеть
    model_deep = MLP(input_size, hidden_sizes=[16, 16]).to(DEVICE)


    print("\nБазовая архитектура:")
    print(model_basic)
    print("\nУглублённая архитектура:")
    print(model_deep)
```

```
EPOCHS = 30


print("\n=== Обучение базовой сети (32 нейрона) ===")
history_basic, time_basic = train_model(
    model_basic, X_train_tensor, y_train_tensor,
    X_test_tensor, y_test_tensor, epochs=EPOCHS,
)
print(f"Время обучения: {time_basic:.2f} с")


print("\n=== Обучение углублённой сети (16 + 16 нейронов) ===")
history_deep, time_deep = train_model(
    model_deep, X_train_tensor, y_train_tensor,
    X_test_tensor, y_test_tensor, epochs=EPOCHS,
)
print(f"Время обучения: {time_deep:.2f} с")


metrics_basic, pred_basic, real = evaluate_model(
    model_basic, X_test_tensor, y_test_tensor
)
metrics_deep, pred_deep, _ = evaluate_model(
    model_deep, X_test_tensor, y_test_tensor
)


comparison = pd.DataFrame({
    "Базовая (32)": {**metrics_basic, "Время обучения, с": time_basic},
    "Углублённая (16 + 16)": {
        **metrics_deep, "Время обучения, с": time_deep
    },
})


print("\nСравнение архитектур (метрики для класса >50K):")
print(comparison.round(4).to_string())


print("\nБазовая модель — отчёт о классификации:")
print(classification_report(
    real, pred_basic,
    target_names=["Доход <=50K", "Доход >50K"],
    zero_division=0,
))


print("Углублённая модель — отчёт о классификации:")
print(classification_report(
    real, pred_deep,
    target_names=["Доход <=50K", "Доход >50K"],
    zero_division=0,
))


plot_loss_curves(history_basic, history_deep, EPOCHS)
plot_confusion_matrices(pred_basic, pred_deep, real)
plot_metrics_comparison(comparison)


best_accuracy = comparison.loc["Accuracy"].idxmax()
best_f1 = comparison.loc["F1-score"].idxmax()
delta_accuracy = metrics_deep["Accuracy"] - metrics_basic["Accuracy"]
delta_f1 = metrics_deep["F1-score"] - metrics_basic["F1-score"]


print("\nРЕЗУЛЬТАТЫ ЭКСПЕРИМЕНТА")
print(f"Точность базовой сети:      {metrics_basic['Accuracy']:.4f}")
print(f"Точность углублённой сети:  {metrics_deep['Accuracy']:.4f}")
print(f"Изменение точности:         {delta_accuracy:+.4f}")
print(f"F1-score базовой сети:      {metrics_basic['F1-score']:.4f}")
print(f"F1-score углублённой сети:  {metrics_deep['F1-score']:.4f}")
```

```
    print(f"Изменение F1-score:         {delta_f1:+.4f}")
    print(f"Более точная модель:        {best_accuracy}")
    print(f"Лучшая по F1-score:         {best_f1}")


    comparison.round(4).to_csv(
        os.path.join(REPORT_DIR, "lab4_metrics.csv"), encoding="utf-8"
    )
    print("\nРабота программы завершена.")



if __name__ == "__main__":
    main()
```

Результаты работы программы:

```
Лабораторная работа №4
Вариант 10 — Adult Census Income
Устройство для вычислений: cpu
Версия PyTorch: 2.15.0+cpu


Размер набора данных: (32561, 15)
Количество пропусков: 4262
Распределение целевой переменной:
income
<=50K    24720
>50K      7841


Обучающая выборка: (24420, 104)
Тестовая выборка: (8141, 104)
Число входных признаков после кодирования: 104
Доля доходов >50K в обучении: 0.2408


Базовая архитектура:
MLP(
  (network): Sequential(
    (0): Linear(in_features=104, out_features=32, bias=True)
    (1): ReLU()
    (2): Linear(in_features=32, out_features=1, bias=True)
  )
)


Углублённая архитектура:
MLP(
  (network): Sequential(
    (0): Linear(in_features=104, out_features=16, bias=True)
    (1): ReLU()
    (2): Linear(in_features=16, out_features=16, bias=True)
    (3): ReLU()
    (4): Linear(in_features=16, out_features=1, bias=True)
  )
)


=== Обучение базовой сети (32 нейрона) ===
Эпоха  1/30: ошибка обучения = 0.5112, ошибка теста = 0.3755
Эпоха  5/30: ошибка обучения = 0.3152, ошибка теста = 0.3236
Эпоха 10/30: ошибка обучения = 0.3062, ошибка теста = 0.3189
Эпоха 15/30: ошибка обучения = 0.3010, ошибка теста = 0.3182
Эпоха 20/30: ошибка обучения = 0.2976, ошибка теста = 0.3178
Эпоха 25/30: ошибка обучения = 0.2951, ошибка теста = 0.3175
Эпоха 30/30: ошибка обучения = 0.2933, ошибка теста = 0.3183
Время обучения: 9.10 с


=== Обучение углублённой сети (16 + 16 нейронов) ===
Эпоха  1/30: ошибка обучения = 0.5572, ошибка теста = 0.3774
Эпоха  5/30: ошибка обучения = 0.3146, ошибка теста = 0.3240
Эпоха 10/30: ошибка обучения = 0.3052, ошибка теста = 0.3181
Эпоха 15/30: ошибка обучения = 0.2990, ошибка теста = 0.3184
Эпоха 20/30: ошибка обучения = 0.2950, ошибка теста = 0.3166
Эпоха 25/30: ошибка обучения = 0.2924, ошибка теста = 0.3167
Эпоха 30/30: ошибка обучения = 0.2908, ошибка теста = 0.3173
Время обучения: 8.93 с


Сравнение архитектур (метрики для класса >50K):
                  Базовая (32)  Углублённая (16 + 16)
Accuracy                0.8533                 0.8543
Precision               0.7206                 0.7261
Recall                  0.6383                 0.6342
F1-score                0.6769                 0.6770
```

```
Время обучения, с        9.1007                8.9341

Базовая модель — отчёт о классификации:
              precision    recall  f1-score   support

 Доход <=50K       0.89      0.92      0.91      6181
  Доход >50K       0.72      0.64      0.68      1960

    accuracy                           0.85      8141
   macro avg       0.80      0.78      0.79      8141
weighted avg       0.85      0.85      0.85      8141

Углублённая модель — отчёт о классификации:
              precision    recall  f1-score   support

 Доход <=50K       0.89      0.92      0.91      6181
  Доход >50K       0.73      0.63      0.68      1960

    accuracy                           0.85      8141
   macro avg       0.81      0.78      0.79      8141
weighted avg       0.85      0.85      0.85      8141

Сохранён график:
C:\Users\widuxd\PycharmProjects\ml_as68\reports\Khomich\4\rep\lab4_loss_curves.png
Сохранён график:
C:\Users\widuxd\PycharmProjects\ml_as68\reports\Khomich\4\rep\lab4_confusion_matrices.png
Сохранён график:
C:\Users\widuxd\PycharmProjects\ml_as68\reports\Khomich\4\rep\lab4_metrics_comparison.png

РЕЗУЛЬТАТЫ ЭКСПЕРИМЕНТА
Точность базовой сети:      0.8533
Точность углублённой сети:  0.8543
Изменение точности:         +0.0010
F1-score базовой сети:      0.6769
F1-score углублённой сети:  0.6770
Изменение F1-score:         +0.0001
Более точная модель:        Углублённая (16 + 16)
Лучшая по F1-score:         Углублённая (16 + 16)

Работа программы завершена.
```

Графики и анализ результатов

На рисунке 1 приведены кривые изменения функции потерь на обучающей и тестовой выборках для обеих архитектур. Видно, что потери быстро снижаются на первых эпохах и затем выходят на плато. Тестовая ошибка практически не растёт, то есть выраженного переобучения не наблюдается.

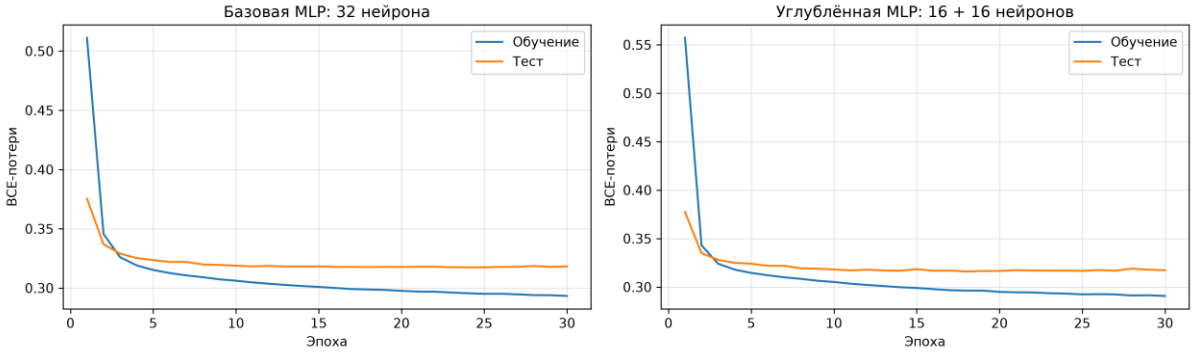


Рисунок 1 — Кривые потерь при обучении базовой и углублённой моделей

На рисунке 2 представлены матрицы ошибок. Обе модели уверенно распознают класс «доход ≤ 50K», однако для более редкого класса «доход > 50K» заметна часть пропущенных положительных объектов (709 и 717 случаев соответственно). Это связано с дисбалансом классов: доля доходов выше 50 тыс. составляет около 24 %.

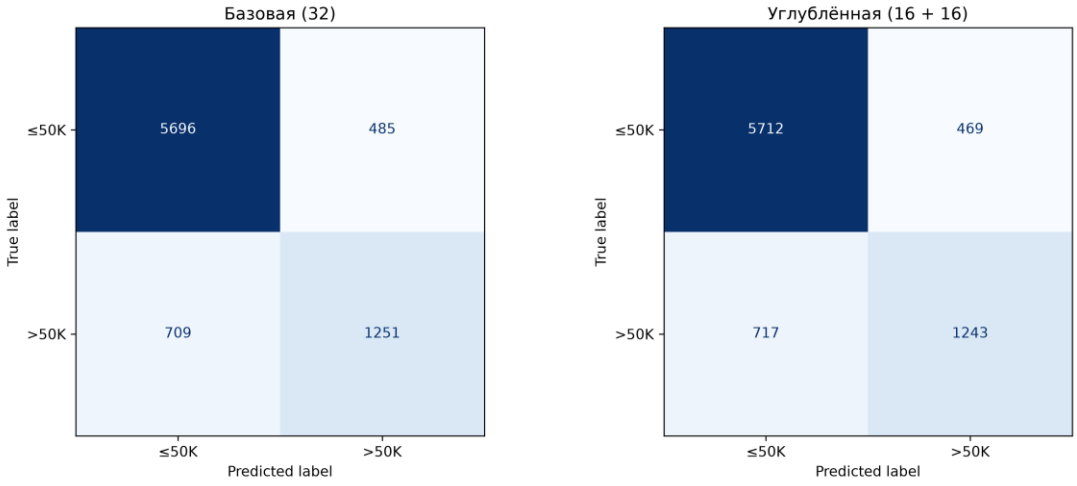


Рисунок 2 — Матрицы ошибок базовой и углублённой моделей

На рисунке 3 показано сравнение основных метрик качества. Обе архитектуры демонстрируют близкие результаты.

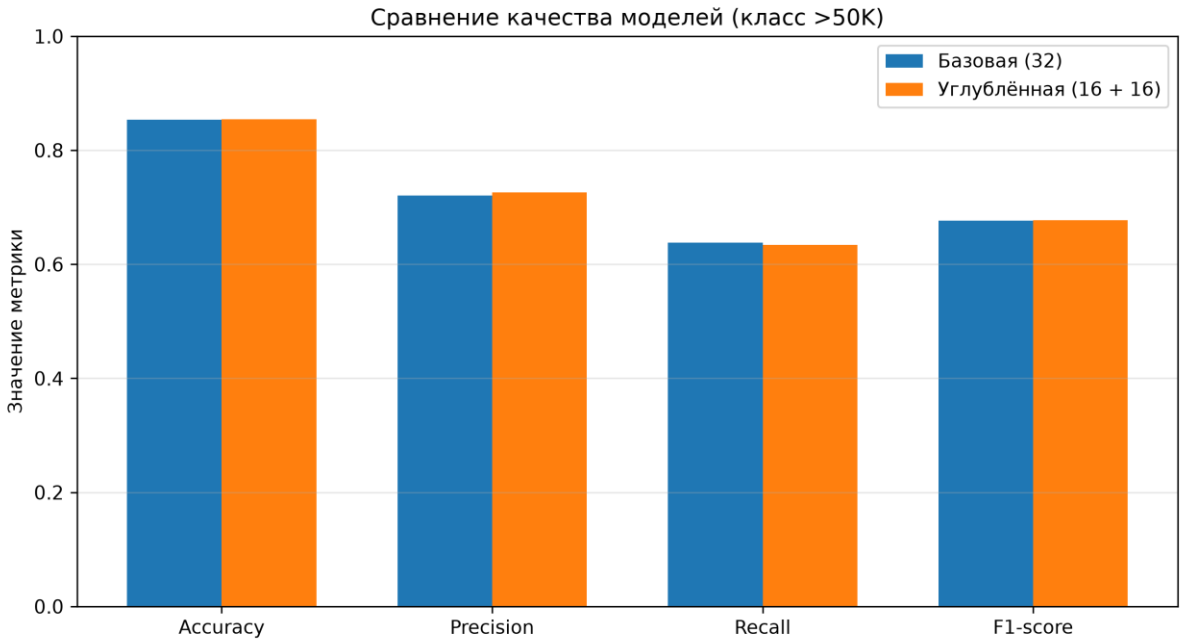


Рисунок 3 — Сравнение метрик качества моделей

Сводная таблица результатов

Метрика	Базовая (32)	Углублённая (16 + 16)
Accuracy	0,8533	0,8543
Precision (>50K)	0,7206	0,7261
Recall (>50K)	0,6383	0,6342
F1-score (>50K)	0,6769	0,6770
Время обучения, с	9,10	8,93

Анализ результатов. Углублённая сеть с двумя скрытыми слоями по 16 нейронов показала точность 0,8543 против 0,8533 у базовой модели с одним скрытым слоем на 32 нейрона, то есть прирост Accuracy составил всего +0,0010 (около 0,1 процентного пункта). Значение F1-score для класса «доход > 50K» практически не изменилось: 0,6770 против 0,6769. При этом Precision немного вырос (0,7261 против 0,7206), а Recall, наоборот, снизился (0,6342 против 0,6383).

Полученный результат показывает, что простое увеличение глубины сети не гарантирует существенного улучшения качества. Для данного датасета обе архитектуры обладают близкой выразительностью, а ограничивающим фактором являются скорее признаки, дисбаланс классов и настройка гиперпараметров, чем количество скрытых слоёв. Базовая модель при этом проще и требует меньше вычислений, поэтому при практически равном качестве она представляется более предпочтительной.

Следует также учитывать, что доля класса «доход > 50K» в данных составляет около 24 %, поэтому метрика Accuracy сама по себе не полностью отражает качество модели. Для такой задачи более информативны Precision, Recall и F1-score по положительному классу.

Вывод

В ходе лабораторной работы был построен, обучен и оценён многослойный перцептрон для задачи прогнозирования уровня дохода по датасету Adult Census Income. Освоены основные этапы работы с нейронной сетью в PyTorch: предобработка данных и кодирование категориальных признаков, преобразование данных в тензоры, определение архитектуры через torch.nn.Module, выбор функции потерь и оптимизатора, написание цикла обучения с прямым и обратным распространением ошибки, а также оценка модели с помощью метрик Accuracy, Precision, Recall и F1-score и матрицы ошибок.

Сравнение базовой архитектуры (один скрытый слой на 32 нейрона) и углублённой (два скрытых слоя по 16 нейронов) показало практически одинаковое качество: прирост Accuracy составил около 0,1 процентного пункта, а F1-score изменился незначительно. Таким образом, увеличение глубины сети не дало ощутимого выигрыша на выбранном наборе данных. Цель лабораторной работы достигнута.